Short Term Undergraduate Software Engineering Training: Course Design and Practice in China

Haoran Yu*, Boyang Yang*, Yaowei Han†, Mengqi Fan†, Wenrui Xu*
*Jisuan Institute of Technology, Beijing Judao Youda Network Technology Co. Ltd., Beijing, China
{yuhaoran, yangboyang, xuwenrui}@jisuanke.com
†College of Software, Nankai University, Tianjin, China
{1511487, 1511484}@mail.nankai.edu.cn

***Abstract*—Modern software engineering requires individuals to work together under well-defined software engineering process as a team. However, there is a gap between what students learn and what industry does. This paper presents a short term project-based software engineering training as a supplement to traditional software engineering courses in China. Students are asked to learn and practice using tools and technologies for software development under the instruction of senior engineers coming from the industry in a week. Then three iterations of group project development, coordinated by moderators, take place in an online-offline combined manner. Cross-testings between groups are applied at last. The course was adopted by Nankai University, China in 2017 and 2018. The quantitative evaluation results and feedback from students showed that the short term training was helpful for undergraduate students in China to understand and meet the requirements of becoming qualified software engineers.**

***Keywords*—software engineering course, team project, course design, China**

I. Introduction

With the development of information technologies, being able to meet the complex software engineer demands becomes significant. However, there is a huge gap between what students learn and what industry does [1]–[3].

To bridge the gap, instructors introduced case-based, project-based and studio-based approaches to software engineering courses [2], [4]–[6]. In 2003, Reichlmayr presented the effectiveness of the agile approach in software engineering course projects [7]. In the last decade, with the prevalence in the industry, agile approach has been adopted by more software engineering courses [8]–[12].

In contrast, according to O'Leary et al. and Li et al., the phenomenon of emphasizing more on theoretical materials than practical experience does not shift much in China [13], [14]. Li et al. pointed out that it was because of the policy constraints in China National Higher Education Catalog approved by the Ministry of Education of China. As further proof, recent works related to software engineering course design published by researchers from China were still introducing project-based and case-based learning as innovative practices in the course [15]–[18].

Meanwhile, some top universities in China have kept pace with the global software engineering education community. Bai et al. integrated version control, project management, DevOps tools into an educational system for agile software engineering projects and provided quantitative views for both instructors and students [19].

Considering the limited practice class hours for software engineering courses in many universities in China, in this work, we demonstrate a design of short-term undergraduate project-based software engineering course. There are three activities in the course that have rarely been introduced in similar courses in the past:

- Students are asked to write a survey about possible third-party services and packages and to trade off between importing them as dependencies or implementing functions from scratch.
- Besides a sprint review meeting within a project group, students are asked to demonstrate in front of all students in different groups who share the same clients at the end of each sprint.
- Considering the cognitive limitations of students within a group, students in different groups who share the same clients are asked to perform as reporters and find out defects in the projects of others.

The course has been applied to software engineering major juniors of Nankai University, China in the summer semesters of 2017 and 2018. The students had the basic knowledge of programming, data structure, and database system but have never been involved in any project development before taking this course.

The efficacy of the course design is measured by quantitative data and qualitative feedback. According to the result, this course enhanced students in both technical abilities and soft skills. Based on the feedback of 2017, we adjusted the course design in 2018 and brought a clear improvement in the outcome.

II. Course Design and Deployment

The course is designed for 3rd-year university students in China who have been trained to write short and straightforward programs but without experience in software development and project management. It is intended to help students understand the software development process, to do agile software development and to use tools correctly. And the course duration is restricted to five weeks by the academic calendar.

TABLE I
PROJECT SELECTION

Name	Roles	Sub-systems	Year
Programming Education Game	Children / Parents	Instructional Game / Game Editor	2017
Customer Messaging Platform	Managers / Customers / Service Staffs	Service Desk / Chat bot / Developer Center	2017
Live Education Platform	Managers / Instructors / Students	Staff Management / Course List / Live Room	2017
Teaching Administration System	Managers / Salespersons / Instructors / Parents / Accountants	Registration / Course Management / Data Recording and Analysis	2018
English Learning Application	Instructors / Students / Parents	E-books / Community / Content Management	2018
Science Education Platform	Editors / Students	Live Audio Books / Content Management	2018

A. Prerequisite Techniques Training

Because of the lack of software engineering experience, students are not used to the development environment, programming language, frameworks they need for project development.

In the first week of the course, students take the classes on prerequisite techniques taught by experienced engineers from enterprises every day. Four or five hours of lectures related to web development techniques, version control tools, project management system, refactoring and coding style guidelines are given in the morning and afternoon. After that, assignments are given to students to help them master the content learned at daytime. The assignments are due before the next day's lectures.

Training on prerequisite techniques helps students gain fast-learning ability and imitates the relationship between mentors and interns in enterprises. As a supplement to daytime lectures, considering the differences in students' ability, office hours are hosted by engineers to solve questions from students after lectures until 10 p.m.

B. Projects Selection and Group Formation

Hands-on experience is necessary for enhancing the practical ability of students in the course. Real projects from industrial clients, free software projects and open source projects are widely adopted by similar courses [20], [21].

In 2017, Vincent pointed out that students could understand the requirement better if they are involved in projects related to familiar scenarios, for instance, education tools [22].

As to find suitable real projects related to familiar scenarios, links with enterprises in the education industry are established. By communicating with managers of the enterprises, they show their need in education information systems, cognitive tools, and customer management systems. In accordance with the requirements for involving at least 2 different roles, 3 to 5 subsystems, and cross-platform use, three projects are finally selected each year as shown in Table I.

Groups of 3 to 5 members are then formed, mainly according to the will of students. Take into account the idea of Bernd, Stephan, and Lukas that balanced group is good for software engineering project course, the formation of a group is further constrained with respect to gender and performance in prerequisite techniques training. It is ensured that each group has at least one male student and one female student. Also, it is guaranteed that each group has at least one student whose performance is excellent in prerequisite techniques training.

Then students are asked to score the projects from 1 to 3 in keeping with short descriptions about selected projects. The scores indicate the willingness of getting involved in the projects. The groups are evenly assigned to each project, and the final assignments are decided for the sake of maximizing the mathematical expectation of $3N_3 + 2N_2 + N_1$, in which N_i is the number of groups assigned to a project that scored i by the group.

C. Requirement Analysis

Real projects provide not only functional requirements from real clients but also contexts, constraints and other aspects related to the software. A lecture is given to students to introduce the requirements acquisition process and requirement management tools.

In 2017, students assigned to the same project were asked to sit together with simulated clients who represented real clients and states a simplified version of the requirement.

In 2018, real clients are invited to the classroom and to directly state their needs to the students. However, unlike simulated clients, the expressions of real clients are much more equivocal, paradoxical, and misleading. In this process, 4th-year undergraduate students who finished the course in 2017 were invited to work as moderators to drop hints to students and encourage them to discover vague statements during the communication.

After that, each group is asked to use requirement management tools, including user stories, mind maps, and prototypes to tease out the software requirement [10], [23], [24].

D. Open Source Understanding and Third-party Dependencies

Nowadays, software systems are more elaborate than ever before. Experienced developers choose not to implement all features from scratch. Instead, reusing third-party open source projects as dependencies becomes a more common practice [25].

A lecture is given to students to help them understand how open source community works and how to find stable third-party open source libraries that match the requirements. The lecture also highlights the importance of dependency management techniques and legal issues related to open source licenses.

Each group is asked to finish a survey report about possible selections of third-party open source projects or third-party online services according to the requirements. In addition, they have to argue and decide, for each part of the project, whether they would develop from scratch or reuse a third-party library or package as a dependency.

E. Agile Software Development

Agile software development approach is widely adopted in software development and software engineering course. Scrum is used as a framework in this course.

Git[1] is introduced for source code management and version control. Gitlab[2] is used for project management.

Before writing any code, students are asked to set up three milestones corresponding to three sprint goals on the Gitlab.

Sprint planning meetings are held at the beginning of each sprint to identify what to do for the sprint. Issues are created during the meeting to define what backlog items are selected to be completed in the current sprint. In addition to the description of the issue, deadlines, assignees and time estimation are explicitly appended to the issue.

Members of each group are allowed to start the development after the sprint planning meeting. The student who is responsible for the issue needs to create a new feature branch and checkout the branch locally before starting to code. When a certain amount of work is completed, the modifications should be committed to the feature branch. The student should record time spent for the modifications on related issue pages.

If an issue is resolved on a feature branch, students are asked to push the branch to the remote repository and create a merge request from the feature branch to the master branch [26]. In software engineering, it is widely recognized that a merge should happen after related merge request is thoroughly reviewed [27]. Students are asked to leave positive feedback or point out problems on merge requests. A merge request is counted as reviewed if it has at least one feedback from a member other than the person who creates the merge request.

A milestone, corresponding to a sprint, should be closed after developers resolve all issues scheduled.

The complete development process is shown in Figure 1.

F. Testing and Software Quality

Unit test and coding style are emphasized by real clients as significant requirements. Students are asked to add unit tests to the project and to use linting tools to avoid bad coding style.

The pre-commit hook of Git and Gitlab pipeline are introduced as triggers to unit test and linting tools. The use of these tools ensures the quality of code in both local and remote repositories.

Lectures about interface design, usability, and security of software are given to students in the third week to help students realize and overcome non-functional shortcomings in their projects.

[1]https://git-scm.com

[2]https://www.gitlab.com

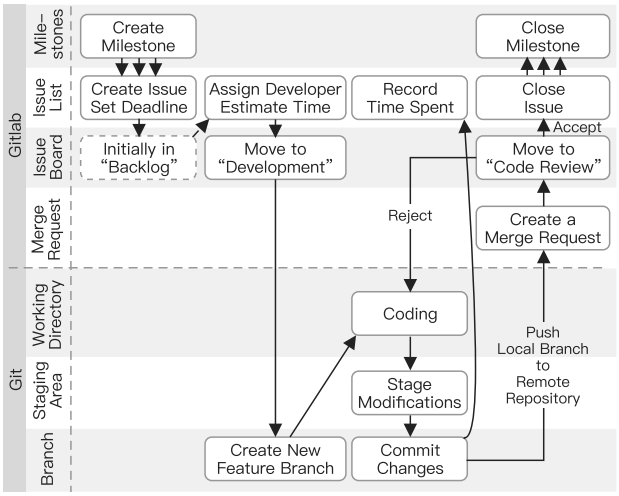


Fig. 1. Development process on Git and Gitlab.

Besides, as an essential part of software engineering, documentation is also required.

G. Defects Finding Between Groups

After the third sprint, each group has substantially completed the project. The codes, documentation, and configuration files for projects are placed in the repositories on the Gitlab.

Subsequently, an activity called cross-group defects finding is launched. Each repository is opened to two other groups who have the same client. The students in the two groups are required to act as reporters and check the projects specified to them.

In the process, if a reporter discovers any functional failure or coding style problem, he will report the problem by creating an issue. The developers of the project need to respond to the issue promptly and fix confirmed defects.

H. Discussion and Demonstration

When each sprint is completed, besides of sprint review meeting, students are scheduled to demonstrate in the classroom. Each group of students has to show what they have done in the past sprint and share the knowledge they gained within 15 to 20 minutes.

The demonstrations are not only intended to enhance students' ability to show and express, but also to provide them with a stronger sense of accomplishment.

At the same time, when a group is presenting, other groups that develop the same project are invited to participate. Discussion and communication between groups are means to promote mutual learning between groups and also introduces a certain sense of competitive pressure.

III. Evaluation Methods

The effectiveness of this course is evaluated based on quantitative measurement and qualitative feedback.

Individual and group performance can be monitored through data collected on Gitlab. These data allow us to see the progress of the students while completing the project.

A. Issue

Gitlab issue management is used to manage backlog tasks in each sprint. Three quantitative indicators related to issues are considered in the evaluation.

- The percentage of issues closed before deadlines: it reflects students' ability on scheduling.
- Percentage of issues with time estimation: it reflects students' conditions of habit formation. Students often forget or subjectively deliberately not mark time estimation when they have insufficient understanding of the issue.
- The accuracy of time estimation: this indicator is specifically defined as $\frac{T_{\text{estimation}}-T_{\text{spent}}}{T_{\text{spent}}}$ where $T_{\text{estimation}}$ is the time estimation of issue and T_{spent} is the actual time spent on the issue.

B. Commit

Commits provide information about who completed what operations at what time, enable tracking on the entire process of development.

Requirements on commit size, commit frequency, and explicit commit messages are confusing for beginners. Developers who are new to software development tend to commit many changes at once with short and unclear commit messages, preventing other developers from correctly understanding what has been done through the commit logs.

In the evaluation, the following two indicators were examined.

- Ratio of I_i to C_i. I_i is the average number of issues closed per day in sprint i. C_i is the average number of commits per day in sprint i.
- Percentage of eligible commit messages that meet the given rules. Each commit must be a complete statement, beginning with an uppercase letter and associated with a specific issue via issue number.

C. Branch and Merge

Branch is an excellent tool for collaborative development. When a developer starts to resolve an issue, he or she has to create a new branch and develops on it. When an issue is resolved, the corresponding branch will be merged into the master branch through a merge request.

Similar to the evaluation of commits, the use of branches can be analyzed as two indicators according to the logs of merge activities.

- Ratio of I_i to M_i. I_i is the average number of issues closed per day in sprint i. M_i is the average number of merges to master branch per day in sprint i.
- Percentage of appropriate merge operations that meets the given rules. Merge records of the master branch into any feature branch will be treated as an error. Merge records that not related to master branch will not be considered eligible in our setting.

D. Teamwork

Teamwork is a vital social aspect in modern software engineering [28], [29]. Relationships between group members may affect the productivity of the group. Good collaboration can help individual learn from others and improve their abilities.

The difference between the number of commits of each member and the average number of commits of all members within a group is considered as a quantitative representation of teamwork.

Besides, at the end of each sprint, students are asked to give internal evaluation scores to other members of their group based on the performance of others.

E. Questionnaire

In addition to quantitative analysis, a questionnaire was given to each student at the end of the course.

In the questionnaire, students are required to express their acceptance of the following statements by rating a score from 1 to 5 and leave additional comments if they have any. Greater number of a score represents stronger agreement.

1) I am familiar with the code version management tools now.
2) I agree more on the importance of software requirements analysis now.
3) I believe more that software development process is helpful to the effectiveness of development now.
4) I realize the importance of teamwork throughout the course.
5) After training, I am more aware of the importance of coding style and software design.
6) I believe more on the significance of testing after training.
7) I tend to write more documentation in software development after training.
8) Weekly presentations allow us learn new knowledge from others.
9) Cross-group defects finding make me aware of our shortcomings and where we can make further improvements.
10) I will tend to survey and trade off between reuse third-party dependencies and coding from scratch in the future.

IV. Results and Analysis

This course was adopted by Nankai University, China in 2017 and 2018. Based on data generated on the Gitlab in the two years, this study illustrates the following results and analysis.

A. Evaluation Results

In the analysis of issues, as the development of the project progresses, there is little change in the percentage of groups which complete over 90% of issues on time. But the percentage of groups which complete less than 70% of issues on time decreases.

There was no significant improvement in the accuracy of time estimation throughout the three sprints in general.

However, the number of issues with an excessive discrepancy in time estimation was decreasing. The time estimation reflects how a developer understands the issue and his judgement of self ability on handling the issue. The reduction of significant deviations in time estimation reflected the improvement in students' ability to weight issues and accurately assessed their own ability.

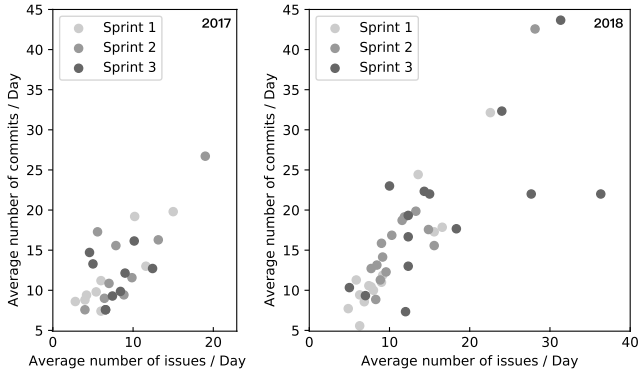


Fig. 2. Relation between issues and commits.

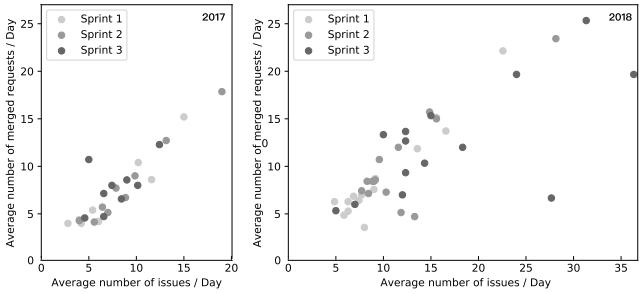


Fig. 3. Relation between issues and merges to master branch.

Students have learned the use of issues, commits and merge requests in a proper way. As shown in Figure 2, points in the figure correspond to the relation between the average number of issues closed per day and commits per day within a sprint. The relation is linear and the slope is not very large. As visualized in Figure 3, the average number of issues closed per day and the average number of merges to master branch per day show linear relation with a slope near to one. The results showed that the students were using an proper commit size and formed expected habits to merge a feature branch into master once for only one issue.

At the same time, there are a few outliers in the above two results. These outliers indicate the existence of some students who modified the project to finish a large number of issues in one commit or in one merge request. Interestingly, the groups corresponding to these outliers in the two results are the same. When communicating with the students in two groups corresponding to outliers in 2018, the students said they knew that they were wrong, but with limited time requirements, they hoped to save the time through the wrong way in the last sprint, so as to ensure that the project could be completed at least on time. This reflects that some students still regard development workflow as a burden. In the future, it is necessary for instructors to further emphasize the long-term value of development workflow earlier, rather than finding the problem until the end.

In the analysis of commit messages and merge, the proportion of students' commit messages that meet the requirements increased in every sprint. And merge errors appeared less and less over time. However, after the launch of cross-group defects finding, an exception had occurred. The proportion of commit messages that meet the requirements declined. We believe that this phenomenon is because of the time pressure during cross-group defects finding and should be further resolved in the future.

As a process for ensuring software quality, code review records were also tracked and analyzed. As the result shows, the number of groups that followed the requirement to merge after code review increased as time goes.

As show in Figure 4, at the beginning of the project, some students committed far exceeded the average of other students in the group. As the project progresses, the difference in the number of student commits in the group gradually converged.

At the same time, as can be seen in the lower-left portion of Figure 5, not only the variance of commits decreased but also the variance of the internal evaluation scores of each group is smaller in the later sprint. It reflects that with the enhancement of engineering ability and coordination, the coherence in groups is boosted synchronously.

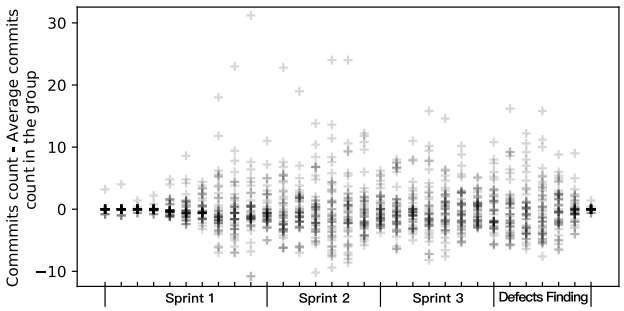


Fig. 4. Teamwork represented by commits.

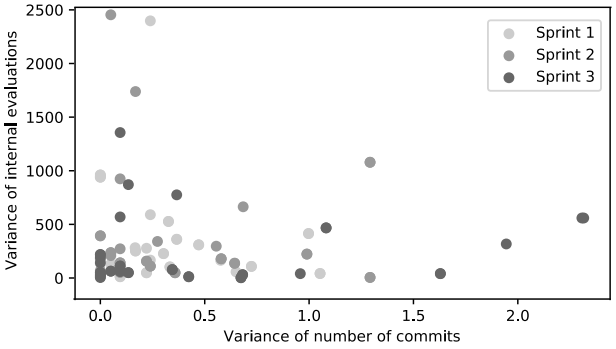


Fig. 5. Teamwork represented by the relationship between internal evaluation and commits.

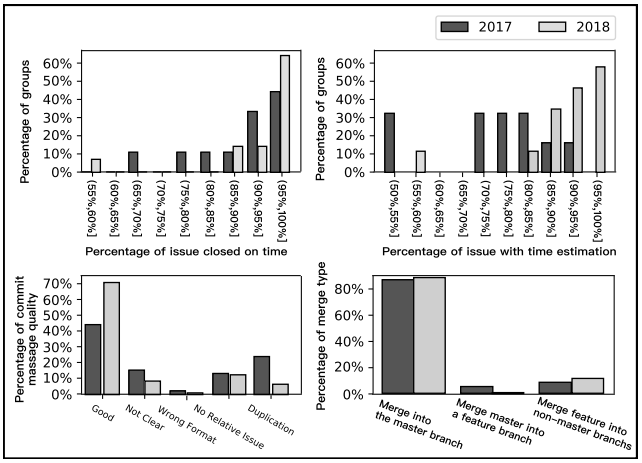


Fig. 6. Improvement from 2017 to 2018

Based on shortcomings reflected in the analysis of course in 2017, additional suggestions for the use of issues, commits, and merge requests were given to students in 2018.

In 2018, the time estimation of each issue was required to be checked by at least two people before development. A "commit-msg" Git hook was recommended to students to check the eligibility of the commit message of each commit. At the same time, the teams were suggested to enable "Merge commit with semi-linear history" on the Gitlab to eliminate the possibility of merge errors being recorded in the log of the master branch.

Through the above data-driven improvements, compared to the results of 2017, the results of 2018 were better regarding the percentage of issues completed on time, the portion of issues with time estimation, quality of commit messages and the use of merge (Figure 6). Also, compare to the result of 2017, in 2018, the points are going to the upper right as time goes in Figure 2 and Figure 3. This result indicated that students' ability to disassemble issues was gradually improving.

B. Feedback from Students

All students filled out the questionnaire within one week after the course. 42 samples in 2017 and 70 samples in 2018 were collected. The feedback of scores can be referred in Table II.

TABLE II
AVERAGE SCORE GIVEN BY STUDENTS

Year	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
2017	4.48	4.60	4.79	4.43	4.81	4.52	4.40	4.62	4.24	4.52
2018	4.53	4.86	4.80	4.50	4.80	4.46	4.49	4.57	4.47	4.53

The results of the questionnaire illustrated the overall effectiveness of the course.

However, 2017 data reflected a certain problem with the design of the cross-group defects finding activity. After talking with some of the students, we found that when the students knew that the group they were checking was checking them, they would communicate privately, try to exchange benefit, and seek ways to reduce the workload of each other. Therefore, in 2018, when this activity was carried out, it was ensured that there was no mutual check between the groups, and the effectiveness of this improvement is presented by the results of the questionnaire in Table II.

In terms of training in software requirements acquisition, as mentioned in previous sections, different approaches are adopted in 2017 and 2018. The results of the questionnaire highlight the importance of real clients. Allowing students to face real clients and process raw expressions from clients is a better practice to help students understand the importance of software requirements analysis.

In addition, there are some problems exposed by the results of the questionnaire that should be resolved in the future. About one-tenth of the students gave a score less than 3 to question 6 and question 7, indicating that they are still not aware of the importance of software testing and software documentation after training.

Furthermore, there are some students who still can't understand the importance of teamwork after they finish the course. 13.9% of students in 2017 and 13.8% of students in 2018 gave a score less than 3 to question 4. Many of these students expressed their dissatisfaction with their teammates in comments. One student in 2018 even wrote, "I would rather work on a project alone than team up with others." Raising students' attention to teamwork in a time-limited course is still a challenging problem that needs to be solved.

V. CONCLUSION

This work demonstrates a short-term undergraduate course design of a project-based software engineering training. Its effectiveness had been verified through the practice in the Nankai University, China in 2017 and 2018. In the practice of 2018, some effective improvements to the course design were given to solve some of the problems found in the practice of 2017.

This course design can be viewed a reference for instructors who wish to offer short-term software engineering training courses within a limited time. The three innovative activities presented in this course design are worth using. The verified significance for direct communication between real clients and students are also worthy of reference for instructors.

There is still room for improvement in the future. More detailed quantitative analysis on software testing and documentation needs to be introduced to judge the outcome of the course. And further enhancement are needed for the timely detection and correction of misconduct exhibited by a few students.

In the analysis of teamwork, as shown in Figure 5, there are two kinds of interesting outliers. A small number of groups had an uneven workload with a harmonious relationship. Another small number of groups had a situation in which the workload of members was balanced, but the variance

of internal evaluation scores was significant. Both of these situations need to be further explored in the future.

REFERENCES

[1] V. Ivanov, A. Rogers, G. Succi, J. Yi, and V. Zorin, "What do software engineers care about? gaps between research and practice," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, 2017, pp. 890–895.

[2] T. Nurkkala and S. Brandle, "Software studio: teaching professional software engineering," in *Proceedings of the 42nd ACM technical symposium on Computer science education*. ACM, 2011, pp. 153–158.

[3] P.-W. Ng and S. Huang, "Essence: A framework to help bridge the gap between software engineering education and industry needs," in *Software Engineering Education and Training (CSEE&T), 2013 IEEE 26th Conference on*. IEEE, 2013, pp. 304–308.

[4] B. Bruegge, S. Krusche, and L. Alperowitz, "Software engineering project courses with industrial clients," *ACM Transactions on Computing Education (TOCE)*, vol. 15, no. 4, p. 17, 2015.

[5] C. N. Bull, J. Whittle, and L. Cruickshank, "Studios in software engineering education: towards an evaluable model," in *Proceedings of the 2013 International Conference on Software Engineering*. IEEE Press, 2013, pp. 1063–1072.

[6] J. Whittle, C. Bull, J. Lee, and G. Kotonya, "Teaching in a software design studio: Implications for modeling education," 2014.

[7] T. Reichlmayr, "The agile approach in an undergraduate software engineering course project," in *Frontiers in education, 2003. FIE 2003 33rd annual*, vol. 3. IEEE, 2003, pp. S2C–13.

[8] V. Mahnic, "A capstone course on agile software development using scrum," *IEEE Transactions on Education*, vol. 55, no. 1, pp. 99–106, 2012.

[9] C. Anslow and F. Maurer, "An experience report at teaching a group based agile software development project course," in *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. ACM, 2015, pp. 500–505.

[10] C. Matthies, T. Kowark, M. Uflacker, and H. Plattner, "Agile metrics for a university software engineering course," in *2016 IEEE Frontiers in Education Conference (FIE)*. IEEE, 2016, pp. 1–5.

[11] J. Campbell, S. Kurkovsky, C. W. Liew, and A. Tafliovich, "Scrum and agile methods in software engineering courses," in *Proceedings of the 47th ACM Technical Symposium on Computing Science Education*. ACM, 2016, pp. 319–320.

[12] J. B. Shepherd, "An agile software engineering course with product hand-off," in *Proceedings of the 2nd International Workshop on Software Engineering Education for Millennials*. ACM, 2018, pp. 86–89.

[13] B. Li, H. Ke, S. Zhou, J. Impagliazzo, and M. Zhang, "Chinese perspectives on it education," in *Proceedings of ACM Turing Celebration Conference-China*. ACM, 2018, pp. 39–46.

[14] C. O'Leary, D. Lawless, D. Gordon, L. Haifeng, and K. Bechkoum, "Developing a software engineering curriculum for the emerging software industry in china," in *19th Conference on Software Engineering Education & Training (CSEET'06)*. IEEE, 2006, pp. 115–122.

[15] X. Ren and Y. Dai, "Discussion on innovative practice teaching model of software engineering," 2017.

[16] Y. Gao and S. Zhang, "To cultivate students' engineering practice ability in software engineering teaching," in *2017 World Conference on Management Science and Human Social Development (MSHSD 2017)*. Atlantis Press, 2017.

[17] L. Du and Y. Han, "Teaching reform and research on the course" project training"," in *2018 International Conference on Advances in Social Sciences and Sustainable Development (ASSSD 2018)*. Atlantis Press, 2018.

[18] Y. Han, "Teaching research and innovative practice on the course" software engineering"," in *2018 International Conference on Advances in Social Sciences and Sustainable Development (ASSSD 2018)*. Atlantis Press, 2018.

[19] X. Bai, M. Li, D. Pei, S. Li, and D. Ye, "Continuous delivery of personalized assessment and feedback in agile software engineering projects," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training*. ACM, 2018, pp. 58–67.

[20] D. Dzvonyar and B. Bruegge, "Reaching steady state in software engineering project courses."

[21] K. Buffardi, "Localized open source collaboration in software engineering education," in *Frontiers in Education Conference (FIE), 2015 IEEE*. IEEE, 2015, pp. 1–5.

[22] V. A. Cicirello, "Student developed computer science educational tools as software engineering course projects," *Journal of Computing Sciences in Colleges*, vol. 32, no. 3, pp. 55–61, 2017.

[23] J. Jaafar, "Collaborative mind map tool to facilitate requirement engineering (re)," in *Proceedings of the 3 International Conference on Computing and Informatics, ICOCI 2011, Bandung, Indonesia, 8-9 June 2011*, 2009.

[24] M. Cohn, *User stories applied: For agile software development*. Addison-Wesley Professional, 2004.

[25] R. G. Kula, D. M. German, A. Ouni, T. Ishio, and K. Inoue, "Do developers update their library dependencies?" *Empirical Software Engineering*, vol. 23, no. 1, pp. 384–417, 2018.

[26] L. Haaranen and T. Lehtinen, "Teaching git on the side: Version control system as a course platform," in *Proceedings of the 2015 ACM Conference on Innovation and Technology in Computer Science Education*. ACM, 2015, pp. 87–92.

[27] S. Sripada, Y. R. Reddy, and A. Sureka, "In support of peer code review and inspection in an undergraduate software engineering course," in *Software Engineering Education and Training (CSEET), 2015 IEEE 28th Conference on*. IEEE, 2015, pp. 3–6.

[28] M. Levy, "Educating for empathy in software engineering course."

[29] C. Raibulet and F. A. Fontana, "Collaborative and teamwork software development in an undergraduate software engineering course," *Journal of Systems and Software*, vol. 144, pp. 409–422, 2018.